

第 4 章：存储管理

2026 版

邹兆年

哈尔滨工业大学
计算学部
海量数据计算研究中心
电子邮件: znzou@hit.edu.cn

2026 春

教学内容¹

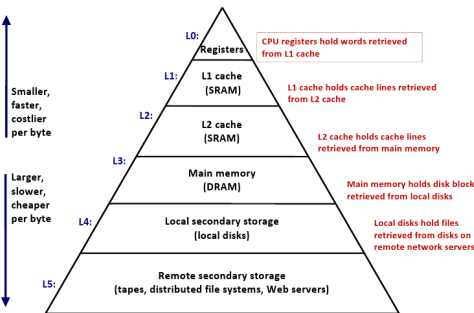
- ① 4.1 存储介质
- ② 4.2 面向磁盘的数据库存储
 - 4.2.1 面向磁盘的数据库存储概述
 - 4.2.2 表文件记录的形式
 - 4.2.3 值的表示
 - 4.2.4 元组记录的布局
 - 4.2.5 页的布局
 - 4.2.6 文件的组织
- ③ 4.3 缓冲池管理
 - 4.3.1 缓冲池管理的基本概念
 - 4.3.2 缓冲池管理器的组成
 - 4.3.3 缓冲池管理器的工作原理
 - 4.3.4 页替换策略

¹课件更新于 2026 年 5 月 6 日

4.1 存储介质

存储层级 (The Memory Hierarchy)

- 越靠上层的存储器离 CPU 越近，容量越小，速度越快，每字节价格越高
- 越靠下层的存储器离 CPU 越远，容量越大，速度越慢，每字节价格越低



存储器的分类

按 CPU 访问存储介质的方式分类

- 主存储器 (Primary Storage)
- 二级存储器 (Secondary Storage)
- 三级存储器 (Tertiary Storage)

主存储器 (Primary Storage)

- 寄存器 (Register)
- 高速缓存 (Cache)
- 内存 (Main Memory)

按字节寻址 (Byte-Addressable)

- CPU 使用 **load/store** 指令直接访问主存储器中的数据

二级存储器 (Secondary Storage)

- 磁盘 (Magnetic Disk) / 机械硬盘 (Hard Disk Drive, HDD)
- 闪存 (Flash Memory) / 固态硬盘 (Solid State Drive, SSD)

按块寻址 (Block-Addressable) + 联机使用

- CPU 无法直接访问二级存储器中的数据，必须使用 **read/write** 指令将数据从二级存储器复制到内存，再进行访问

三级存储器 (Tertiary Storage)

- 磁带 (Magnetic Tape)
- 光盘 (Optical Disk)
- 网络存储 (Network Storage)

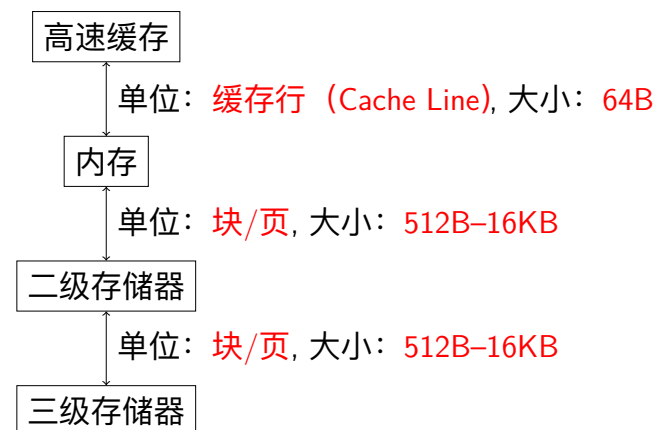
按块寻址 + 脱机使用

- CPU 若要访问三级存储器中的数据，必须先将数据从三级存储器复制到二级存储器

访存时间 (Access Time)

访存时间/ns	存储介质
0.5	L1 cache
7	L2 cache
100	DRAM
150,000	SSD
10,000,000	HDD
30,000,000	网络存储
1,000,000,000	磁带

存储层级之间的数据传输

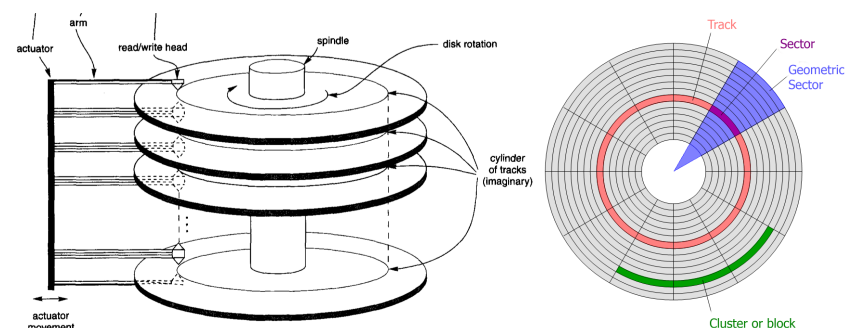


存储器的易失性 (Volatility)

- 主存储器：易失 (Volatility)
- 二级存储器：非易失 (Non-volatility)
- 三级存储器：非易失 (Non-volatility)

磁盘

- 磁盘构件 (Disk Assembly)：扇区 (Sector) ⊂ 磁道 (Track) ⊂ 柱面 (Cylinder)
- 磁头构件 (Head Assembly)：磁头 (Disk Head) 和磁臂 (Disk Arm)



磁盘块 (Block) / 页 (Page)

操作系统在格式化磁盘时将磁盘划分为许多大小相同的块/页

- 磁盘块/页的大小是扇区大小 (通常 512B) 的整数倍
- 磁盘块/页的大小一旦被设置, 则不可以动态改变
- 磁盘块/页的大小通常是 512B–16KB

固态硬盘 (Solid-State Drive, SSD)

构造

- 使用 NAND 闪存 (Flash Memory) 作为存储介质
- 具有与磁盘相同的面向块的接口

特点

- 随机读写速度高于磁盘
- 功耗低于磁盘

SSD 的写操作

特点

- 先擦除, 后写入
- 擦除块通常是 256KB–1MB

缺点

- 写放大 (Write Amplification)
- 寿命低: 擦除次数 100,000–1,000,000 次

解决方案

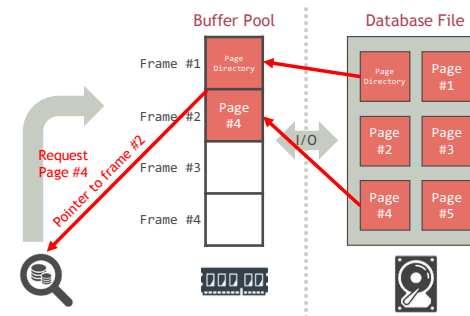
- 逻辑页号 → 物理页号

4.2 面向磁盘的数据库存储

4.2.1 面向磁盘的数据库存储概述

面向磁盘的数据库存储

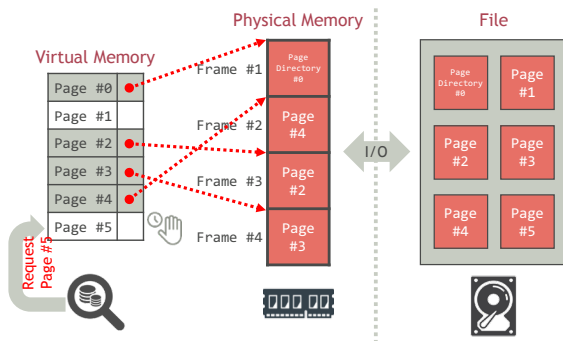
- 数据库持久存储在磁盘文件中
- 读/写数据前先将文件页从磁盘读入内存
- 写数据后将修改过的文件页写回磁盘，替换文件页的原有内容



磁盘存储数据
内存作缓冲区

为何不用操作系统直接管理数据库存储?

- 使用操作系统的内存映射 (Memory Mapping, mmap) 将数据库文件的内容映射到进程的地址空间
- 操作系统负责在磁盘和内存之间移动文件页面



为何该方法不适用于数据库存储管理?

设计目标

- 使 DBMS 能够管理比可用内存容量更大的数据库
- 避免不合理的磁盘 I/O 造成的系统停顿 (stall) 和性能下降

空间方面的设计

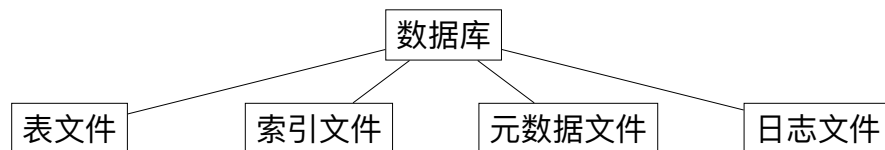
- Q: 将页写到磁盘中什么位置?
- A: 尽量将常在一起使用的页在物理上相近地存储 (数据局部性)

时间方面的设计

- Q: 何时将页读入内存? 何时将页写回磁盘?
- A: 最小化磁盘 I/O 造成的系统停顿次数

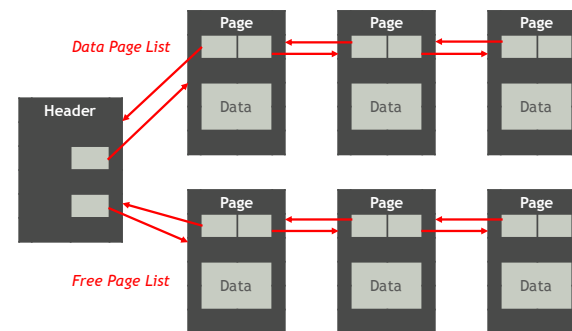
数据库文件

- 数据库存储在一个或多个文件中
- 文件可以存储表、索引、元数据、日志等



文件页

- 一个文件由多个页构成
- 页的大小通常是 512B–16KB
- 每个页拥有唯一的页号 (Page ID)



文件记录

- 文件的每个页包含若干记录
- 记录可以存储行 (元组)、列 (属性)、索引项或日志记录
- 记录由属性值构成



数据存储的粒度



磁盘管理器 (Disk Manager)

- 打开/关闭文件
- 读/写文件页
- 分配/回收文件页

存储管理器 (Storage Manager)

- 读写文件页中的记录（元组、索引项、日志记录等）
- 记录文件页中的空闲空间

4.2.2 表文件记录的形式

表文件记录的形式

- 记录存储表的行
- 记录存储表的列

面向行的数据存储 (Row-Oriented Data Storage)

将一个元组的所有属性值存储在一条记录中

- 当查询仅涉及少量属性时, 也必须读无关的属性值 $\Rightarrow$ 浪费 I/O
- 缓存中相邻字节可能包含不需要的属性值 $\Rightarrow$ 缓存效率低
- 将不同类型的属性值存储在一条记录中 $\Rightarrow$ 压缩率低

例 (面向行的数据存储)

CS-001	Elsa	F	19	CS
CS-002	Ed	M	19	CS
MA-001	Abby	F	18	Math
PH-001	Nick	M	20	Physics

```
SELECT Sdept, AVG(Sage) FROM Student GROUP BY Sdept;
```

面向列的数据存储 (Column-Oriented Data Storage)

将关系的每个属性单独存储为记录

- 当查询仅涉及少量属性时, 不需要读无关属性 $\Rightarrow$ 减少 I/O
- 缓存中相邻字节来自于同一个属性 $\Rightarrow$ 缓存效率高
- 相同类型的属性值存储在一条记录中 $\Rightarrow$ 压缩率高
- 可以使用 SIMD 指令执行向量化计算

例 (面向列的数据存储)

CS-001	Elsa	F	19	CS
CS-002	Ed	M	19	CS
MA-001	Abby	F	18	Math
PH-001	Nick	M	20	Physics

```
SELECT Sdept, AVG(Sage) FROM Student GROUP BY Sdept;
```

面向列的存储的缺点

面向列的存储适合联机分析处理 (Online Analytical Processing, OLAP), 不适合联机事务处理 (Online Transaction Processing, OLTP)

- 元组重构代价高
- 元组删除和修改代价高
- 解压代价

例 (元组重构)

CS-001	Elsa	F	19	CS
CS-002	Ed	M	19	CS
MA-001	Abby	F	18	Math
PH-001	Nick	M	20	Physics

```
SELECT * FROM Student WHERE Sno = 'MA-001';
```

4.2.3 值的表示

数的表示

整数 INT/INTEGER/SMALLINT/BIGINT

- 采用 C/C++ 的整数表示

浮点数 FLOAT(n)/REAL/DOUBLE PRECISION

- 采用 IEEE-754 标准的浮点数表示
- 通常比定点数运算快
- 存在舍入误差

定点数 NUMERIC(p,s)/DECIMAL(p,s)

- 表示为可变长二进制序列（含元数据）
- 不存在舍入误差

字符串的表示

定长字符串 CHAR(n)/CHARACTER(n)

- 表示为 n 个字符的数组
- 如果字符串长度小于 n ，则数组中字符串后面的字符用空字符 (null) 补全
- 例：'cat' 用 CHAR(5) 字符串表示为

'c'	'a'	't'	0	0
-----	-----	-----	---	---

变长字符串 VARCHAR(n)/CHARACTER VARYING(n)/CLOB

- 不使用 C/C++ 中以空字符结尾的字符串表示
- 数组头部若干字节存储字符串的长度，随后存储字符串的内容
- 例：'cat' 用 VARCHAR(5) 字符串表示为

3	'c'	'a'	't'
---	-----	-----	-----

字节串的表示

定长字节串 BINARY(n)

- 表示为 n 个字节的数组
- 如果字节串长度小于 n ，则数组中字节串后面的字节用 0 补全

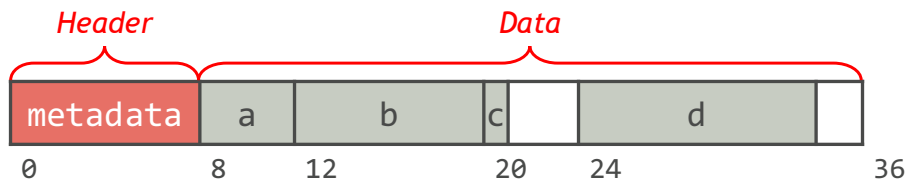
变长字节串 VARBINARY(n)/BINARY VARYING(n)/BLOB

- 表示为可变长字节数组
- 数组头部若干字节存储字节串的长度，随后存储字节串的内容

4.2.4 元组记录的布局

元组记录的布局

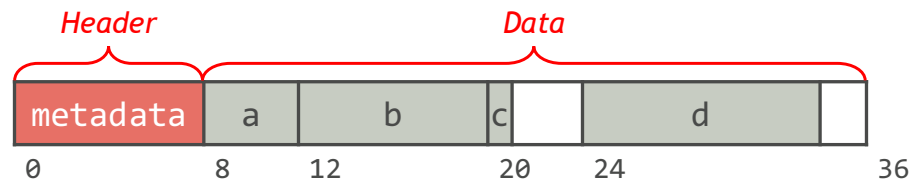
- 头部
- 数据部分



元组记录的头部 (Header)

记录元组的元数据

- 指向元组所在关系的模式定义的指针
- 元组的长度
- 元组的最后修改时间
- 元组的可见性 (与并发控制相关)
- 元组的哪些属性值非空, 用位图 (bitmap) 表示

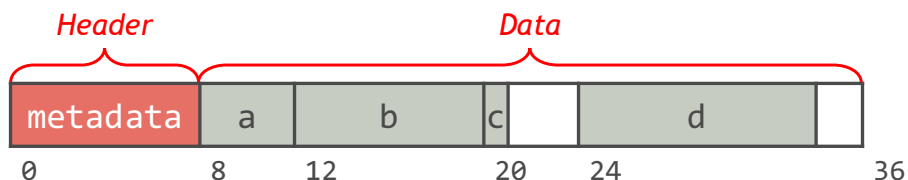


元组记录的数据部分

存储元组属性值

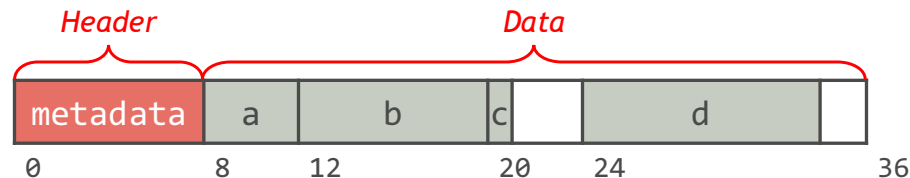
- 通常按关系模式定义的属性顺序存储属性值
- 每个属性值在记录中的存储偏移量 (相对于记录起始位置) 是 **4 或 8 字节** 的倍数

```
CREATE TABLE Foo (  
  a INT NOT NULL,  
  b BIGINT NOT NULL,  
  c CHAR NOT NULL,  
  d VARCHAR(10) NOT NULL);
```



变长元组的记录布局

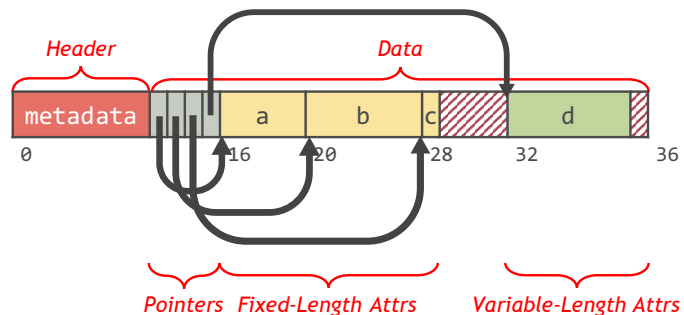
- 按模式定义的顺序将定长属性值和变长属性值混在一起存储不利于数据更新



变长元组的记录布局

定长属性值和变长属性值分离存储

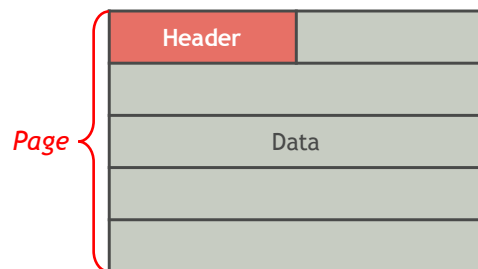
- 定长属性值和变长属性值分别置于元组数据部分的前端和后端
- 元组头后面设置指针数组，其中第 i 个指针指向第 i 个属性值



4.2.5 页的布局

页的布局

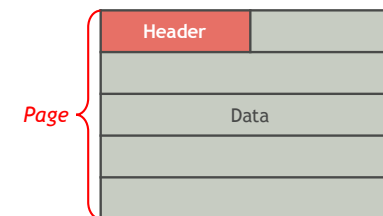
- 页头
- 页的数据部分



页头

记录页的元数据

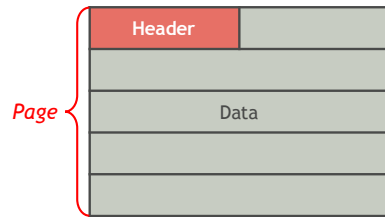
- 页的大小
- 页的校验和 (Checksum)
- DBMS 的版本
- 页的可见性 (与并发控制相关)
- 页的数据部分的压缩方法



页的数据部分

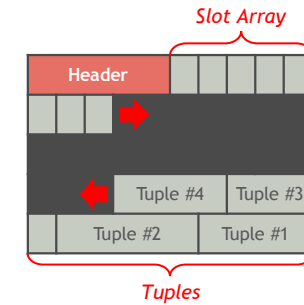
存储元组

- 页中元组数量不固定，长度不固定，元组存什么位置？



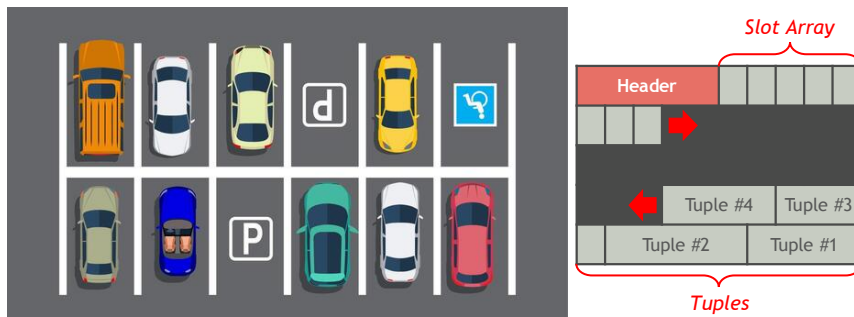
分槽页 (Slotted Page) 布局方法

- 槽数组 (Slot Array)
- 元组记录



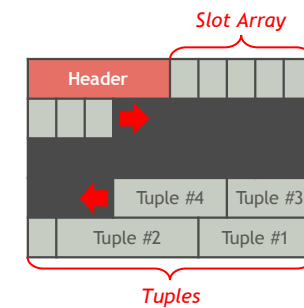
槽 (Slot)

- “规划出来存元组的地方”
- 槽本身不存储元组记录，而是存放该元组记录的偏移量
- 槽中没有元组就空着



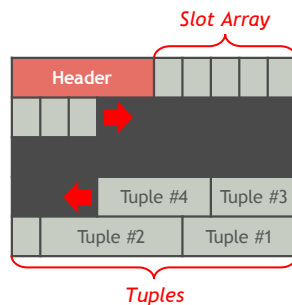
槽数组 (Slot Array)

- 存放在页数据部分的前部
- 向后动态生长



元组记录

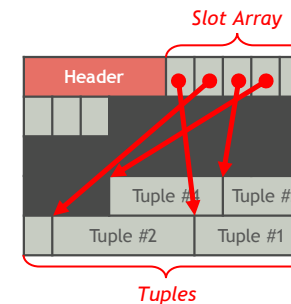
- 存放在页数据部分的后部
- 从后向前依次存放元组记录
- 元组记录存储位置的偏移量是4 或 8 字节的倍数



分槽页的元数据

分槽页的元数据存储在页头中

- 槽的数量：用于确定下一个新增槽的编号
- 存储位置最靠前的元组记录的偏移量：用于确定新元组的存放位置



元组记录的编号 (Record ID)

(页号, 槽号)

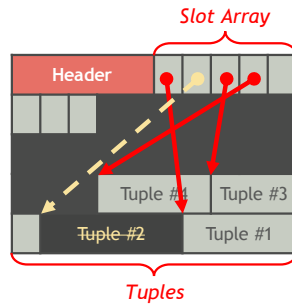
- 页号 (Page ID)：元组所在页的编号
- 槽号 (Slot Number)：元组所在槽的编号

溢出页 (Overflow Page)

- DBMS 基本不允许元组的大小超过页的大小
- 如果一个元组的大小超过页的大小，DBMS 可将元组中 CLOB/BLOB 型属性值存储在溢出页中，并在元组中存储指向溢出页中 CLOB/BLOB 型大对象的指针

页的碎片化 (Page Fragmentation)

- 删除元组记录会产生碎片 (Fragment)
- 浪费磁盘空间, 增加磁盘 I/O
- 定期使用 VACUUM 命令回收空间



4.2.6 文件的组织

文件的组织

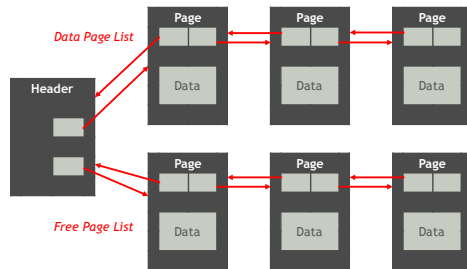
- 堆文件组织 (Heap File Organization): 按任意顺序存放元组
- 顺序/有序文件组织 (Sequential/Sorted File Organization): 按排序键有序存放元组
- 哈希文件组织 (Hash File Organization): 根据元组键的哈希值来确定存放位置

文件页的组织

- 基于链表的文件页组织方法
- 基于页目录的文件页组织方法

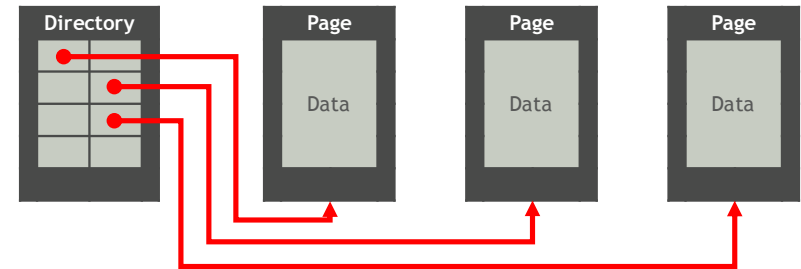
基于链表的文件页组织方法

- **数据页**：存储元组的页，全部数据页组成数据页链表
- **空闲页**：未存储元组的页，全部空闲页组成空闲页链表
- **文件头**：文件的第 1 个页，记录文件的元数据、数据页链表指针（页号）、空闲页链表指针（页号）



基于页目录的文件页组织方法

- **数据页**：存储元组的页
- **页目录 (Page Directory)**：记录每个数据页在文件中的位置和空闲空间
- DBMS 必须保证页目录中的信息与数据页的状态同步

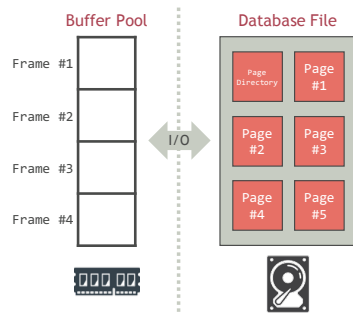


4.3 缓冲池管理

4.3.1 缓冲池管理的基本概念

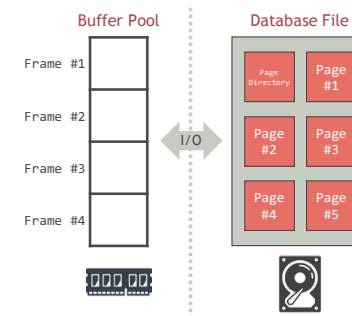
内存缓冲区 (Buffer)

- CPU 无法直接读/写磁盘中的数据，必须先将文件页从磁盘读入内存
- 数据库文件的大小经常超过 DBMS 的可用内存容量
- DBMS 必须用好内存缓冲区



缓冲池 (Buffer Pool)

- DBMS 将内存缓冲区划分为一系列连续的、页大小的单元，叫作页框 (Frame)
- 页框数组称作缓冲池 (Buffer Pool)



缓冲池管理器 (Buffer Pool Manager)

- 将查询处理线程请求读/写的文件页读入缓冲池
- 将缓冲池中某个修改过的页适时写回文件，覆盖原有文件页
- 强制将缓冲池中某个页写回文件，覆盖原有文件页
- 在文件中添加页，并读入缓冲池

4.3.2 缓冲池管理器的组成

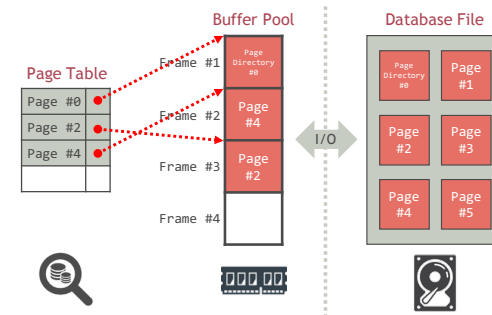
缓冲池管理器的组成

- 缓冲池
- 页表 (Page Table)
- 页框的状态信息
- 空闲页框列表

页表 (Page Table)

页表记录缓冲池中当前存放的页的页号及它们的内存地址

- 页表可以用内存哈希表实现
- 对于缓冲池中存放的页，页表将其页号映射为该页所在页框的内存地址



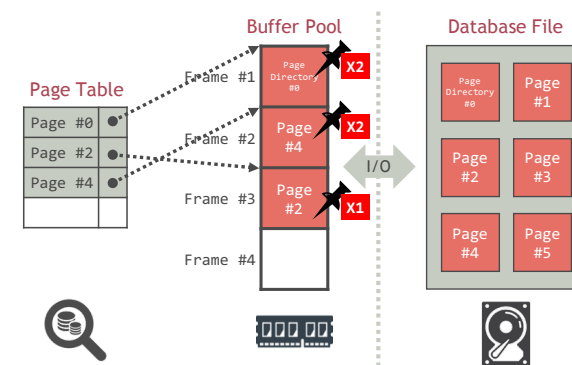
页框的状态信息

缓冲池管理器用两个变量记录每个页框的状态

- `pin_count`
- `dirty`

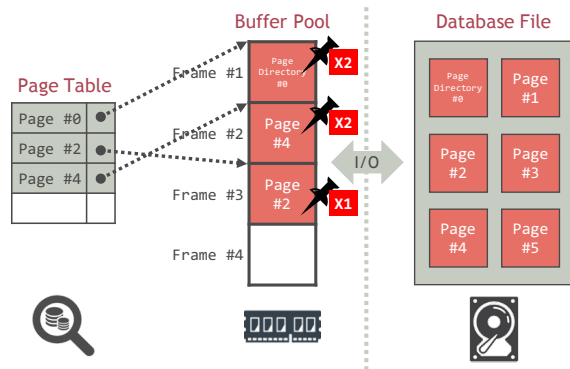
页框的 `pin_count` 变量

- 记录页框中的页当前被请求但尚未释放的次数，即引用计数
- 初始值为 0



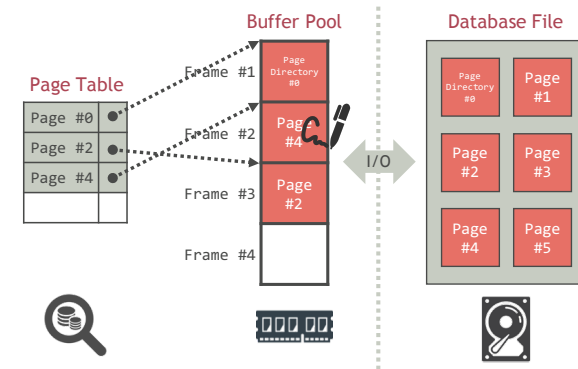
改变页框 pin_count 的基本操作

- 固定 (Pin): 将 pin_count 加 1, 代表多了一个线程请求该页框中的页
- 取消固定 (Unpin): 将 pin_count 减 1, 代表少了一个线程请求该页框中的页



页框的 dirty 变量

- 记录页框中的页被读入缓冲池后是否被修改过
- 初始值为 false



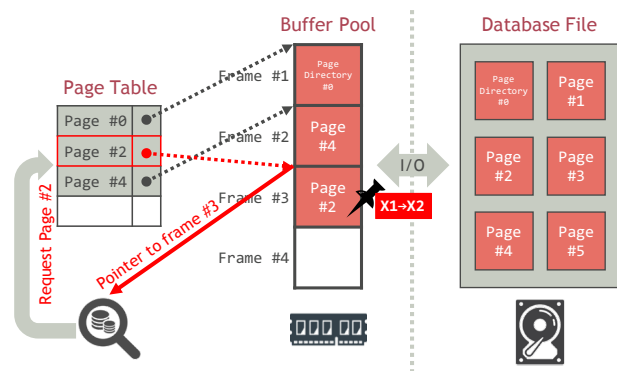
4.3.3 缓冲池管理器的工作原理

缓冲池管理器的功能

- 请求页
- 释放页
- 修改页
- 创建页
- 刷写页
- 预取页

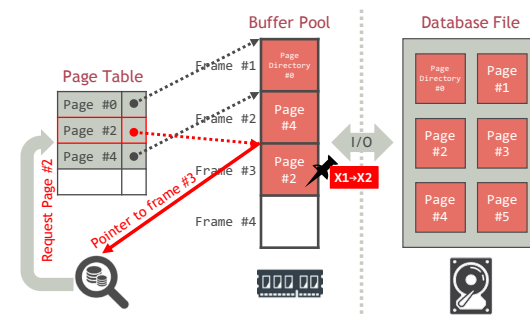
请求页 (Requesting Page)

给定被请求的页的页号, 将该页读入缓冲池, 并返回该页所在页框的地址



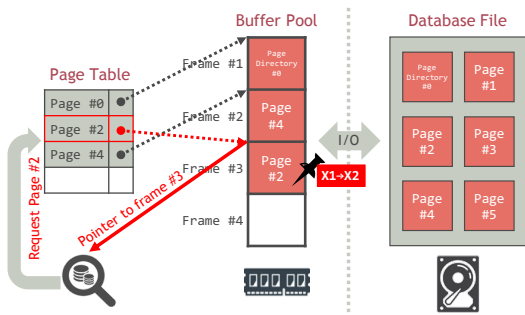
请求页的方法 (情况 1)

- 1 查询页表, 判断被请求的页 P 是否在缓冲池中
- 2 如果 P 在缓冲池中, 则固定 (Pin) 住 P (将 P 所在页框的 `pin_count` 加 1)
- 3 返回 P 所在页框的地址



请求页的方法 (情况 2)

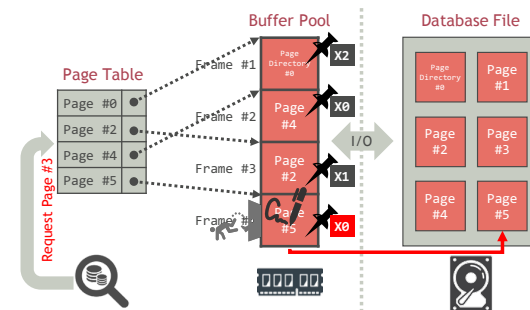
- 1 查询页表, 判断被请求的页 P 是否在缓冲池中
- 2 如果 P 不在缓冲池中, 且存在空闲页框, 则任选一个空闲页框, 通过磁盘管理器将 P 读入该页框, 并固定住 P
- 3 返回 P 所在页框的地址



请求页的方法 (情况 3)

如果 P 不在缓冲池中, 且没有空闲页框, 则执行下列步骤

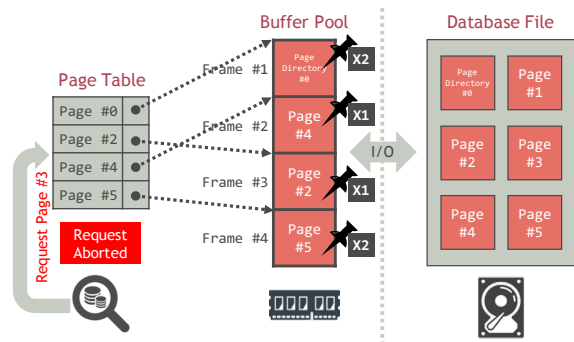
- 1 **选替换页**: 使用缓冲池的**页替换策略** (Page Replacement Policy), 从缓冲池选择一个 `pin_count = 0` 的页框 (代表未在使用), 并固定住该页框
- 2 **处理替换页**: 如果该页框的 `dirty` 值为真, 则将该页框中的页写回文件
- 3 **读请求页**: 将被请求的页 P 读入该页框, 返回该页框的地址



请求页的方法（情况 4）

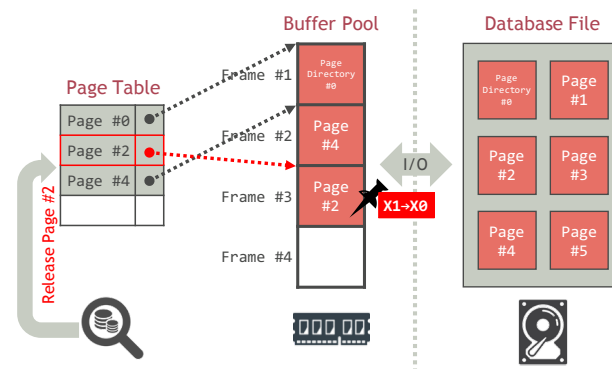
如果缓冲池中没有 `pin_count = 0` 的页框，则请求页的线程开始等待，直至 DBMS 释放了缓冲池中的页面

- 实际上，DBMS 会中止（abort）提出该请求的事务，并重新执行它



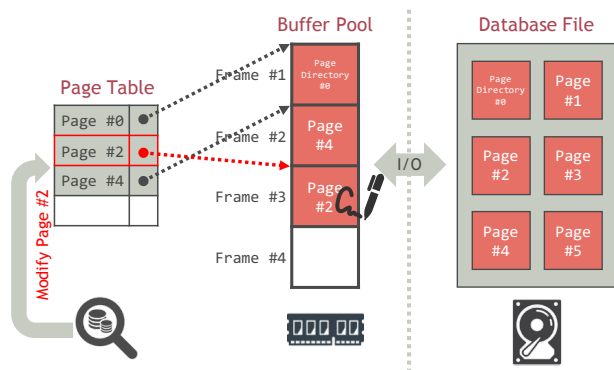
释放页（Releasing Page）

- 一个查询处理线程请求某个页后必须释放这个页，否则缓冲池将耗尽
- 取消固定（Unpin）被释放的页（将该页所在页框的 `pin_count` 减 1）



修改页（Modifying Page）

- 将被修改的页所在页框的 `dirty` 值置为 `true`
- 修改该页的数据



创建页（Allocating Page）

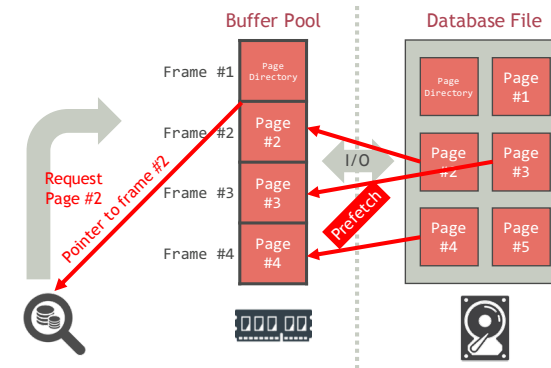
- 通过磁盘管理器在文件中添加一个新的页，并返回其页号
- 使用该页号请求页

刷写页 (Flushing Page)

- ① 通过磁盘管理器将被刷写的页强制写回文件，覆盖原有文件页

预取页 (Prefetching Page)

- 当缓冲区管理器处理页请求时，经常会从文件中预取一些近邻的页（未来可能被请求）
- 如果被预取的页后来真的被请求了，则页请求会更快完成

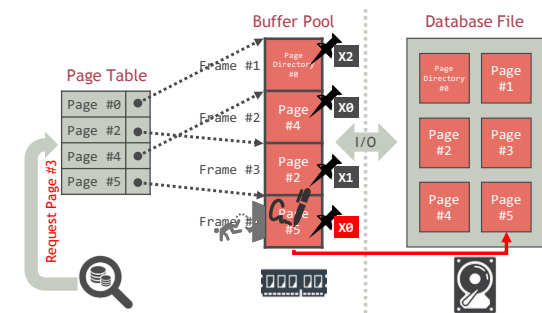


4.3.4 页替换策略

页替换策略 (Page Replacement Policy)

页替换策略对数据库访问延迟的影响很大

- 最近最少使用策略 (Least Recently Used, LRU)
- 时钟策略 (Clock)
- 先进先出策略 (First-In First-Out, FIFO)
- 最近最多使用策略 (Most Recently Used, MRU)



最近最少使用策略 (Least Recently Used, LRU)

最近用过的页，未来还可能再用

- 运行时记录缓冲池中每个页最近一次被访问的时间
- 替换 `pin_count = 0` 且最久未访问的页

LRU 策略的实现

- 用一个链表维护页框的访问顺序
- 每当访问某个页框，就把它移到链表头部（表示最近使用）
- 需要替换页时，直接移除链表尾部的页框（表示最久未被使用）

时钟替换策略 (Clock Replacement Policy)

时钟替换策略是对 LRU 策略的近似

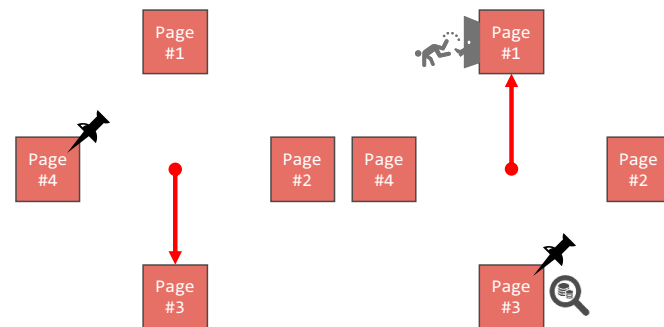
运行时

- 将缓冲池中所有页框组织成一个环形（“表盘”）
- 为每个页框记录一个引用位（Reference Bit），初始值为 0（表示最近未被访问）
- 当某个页被读入某个页框时，将该页框的引用位置为 1（表示最近被访问）
- 当访问某个页框中的页时，将该页框的引用位置为 1（表示最近被访问）

时钟替换策略 (Clock Replacement Policy)

页替换时

- 用一根旋转的“表针”搜索第一个 `pin_count = 0` 且引用位为 0（最近未被访问）的页框，替换该页框中的页
- 如果表针扫过的页框引用位为 1（最近被访问过），则将其引用位重置为 0（最近未被访问）



探究式学习：DBMS 的缓冲池 vs OS 的虚拟内存

借助 LLM 学习

- DBMS 的缓冲池与 OS 的虚拟内存有何相同点？
- DBMS 的缓冲池与 OS 的虚拟内存有何不同点？

总结

- ① 4.1 存储介质
- ② 4.2 面向磁盘的数据库存储
 - 4.2.1 面向磁盘的数据库存储概述
 - 4.2.2 表文件记录的形式
 - 4.2.3 值的表示
 - 4.2.4 元组记录的布局
 - 4.2.5 页的布局
 - 4.2.6 文件的组织
- ③ 4.3 缓冲池管理
 - 4.3.1 缓冲池管理的基本概念
 - 4.3.2 缓冲池管理器的组成
 - 4.3.3 缓冲池管理器的工作原理
 - 4.3.4 页替换策略